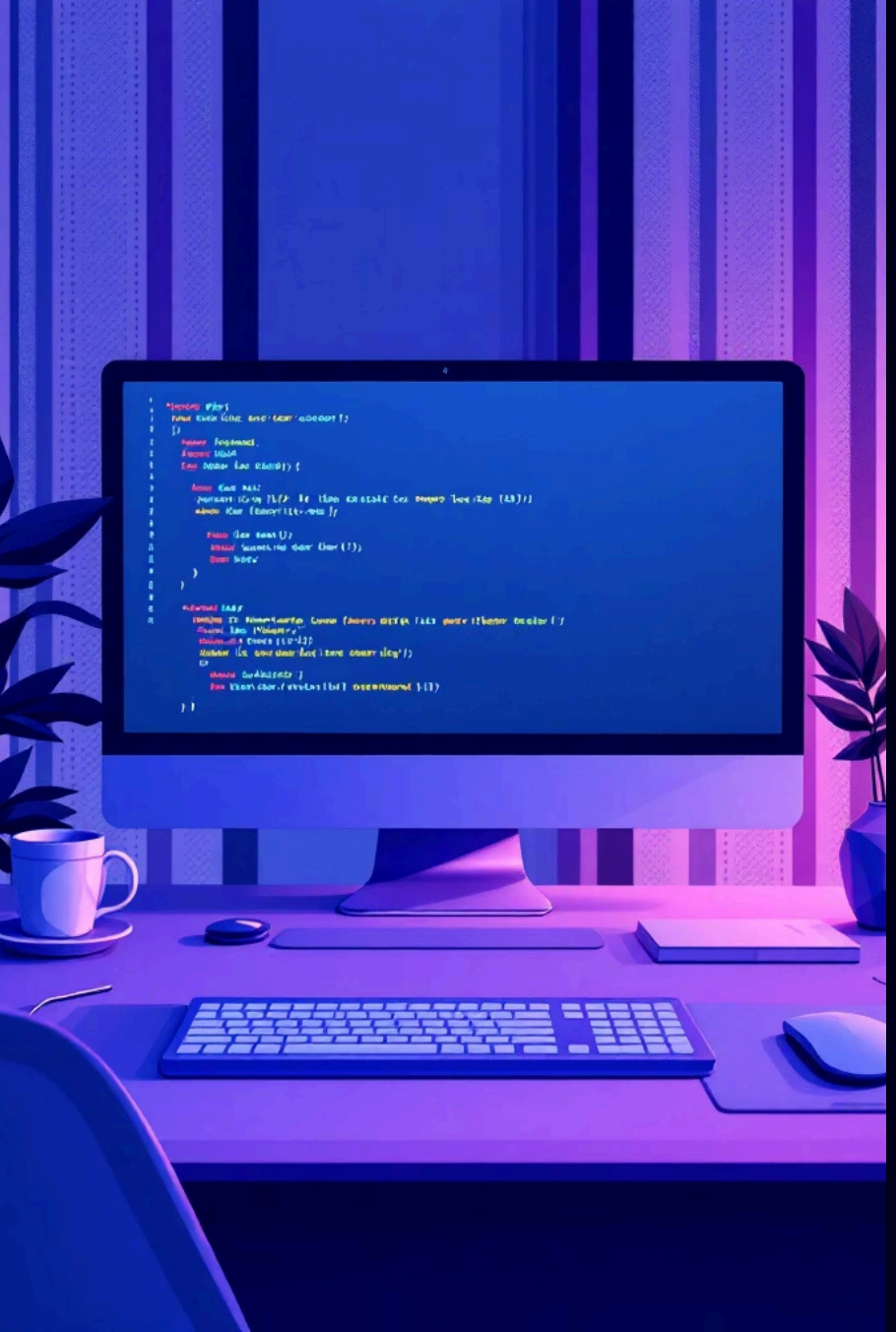

Python: Lógica, Decisões e Repetições

Bem-vindo à Aula 5 da Unidade III! Nesta jornada pelo mundo da programação em Python, vamos explorar os fundamentos que tornam os programas inteligentes e dinâmicos. As expressões lógicas, estruturas de decisão e repetição são os pilares que permitem aos programas tomar decisões, comparar valores e executar tarefas repetidamente.

Estes conceitos são essenciais para transformar código estático em programas interativos que respondem a diferentes situações e dados. Ao dominar essas estruturas, você estará preparado para criar programas mais sofisticados e funcionais.

By Prof. Cloves Rocha

DESAFIO 4: Acesso disponível no GitHub da disciplina. Data de entrega: 06/ABR/2026 (Aula 5)

Made with GAMMA

Desvendando a Lógica em Python

A lógica booleana é o coração da programação. Ela permite que seus programas façam perguntas sobre dados e tomem decisões com base nas respostas. Toda expressão lógica em Python retorna apenas dois valores: **True** (verdadeiro) ou **False** (falso). Essa simplicidade é poderosa!

Operadores Lógicos: O Cérebro das Decisões

Os operadores lógicos combinam ou modificam expressões booleanas, permitindo criar condições complexas e sofisticadas.

Operador AND (&)

Retorna **True** apenas se **ambas** as condições forem verdadeiras.

```
True and True   = True
True and False  = False
False and True  = False
False and False = False
```

Exemplo prático: `idade >= 18 and possui_habilitacao == True`

Operador OR (|)

Retorna **True** se **pelo menos uma** das condições for verdadeira.

```
True or True   = True
True or False  = True
False or True  = True
False or False = False
```

Exemplo prático: `dia_semana == 'sábado' or dia_semana == 'domingo'`

Operador NOT (~)

Inverte o valor lógico: **True** vira **False** e vice-versa.

```
not True  = False
not False = True
```

Exemplo prático: `not (numero % 2 == 0)` verifica se o número é ímpar

Operadores de Comparação: Comparando Valores

Estes operadores comparam dois valores e retornam um resultado booleano, essencial para criar as condições que controlam o fluxo do programa.

Operadores Básicos

- `==` - Igual a
- `!=` - Diferente de
- `<` - Menor que
- `>` - Maior que
- `<=` - Menor ou igual
- `>=` - Maior ou igual

Exemplos Práticos

```
5 == 5    # True
3 != 4    # True
2 < 3     # True
7 > 10    # False
5 <= 5    # True
8 >= 9    # False
```

Precisão e Boas Práticas

Atenção aos Tipos

Comparar tipos diferentes pode causar erros. Use `int()`, `float()` ou `str()` para converter antes da comparação.

Precedência dos Operadores

Use parênteses para controlar a ordem de avaliação: `(idade >= 18) and (salario > 1000)` é mais claro que sem parênteses.

Evite Comparações com Float

Use `math.isclose()` para comparar números de ponto flutuante devido a erros de precisão.

Made with GAMMA

Estruturas de Decisão: O Poder do `if`

As estruturas de decisão são os cérebros dos programas, permitindo que o código tome diferentes caminhos baseados em condições. Sem elas, os programas seriam apenas sequências fixas de comandos. Com `if`, `elif` e `else`, você cria programas inteligentes que respondem dinamicamente aos dados.

O Bloco `if`: A Condição Inicial

O `if` é a estrutura mais fundamental. Ele verifica uma condição e executa um bloco de código apenas se a condição for **True**. É como fazer uma pergunta ao programa: "Se isso for verdade, faça aquilo".

Sintaxe Básica

```
if condição:
    # código executado se a condição
    # for verdadeira
    comando1
    comando2
    # ...
```

Características

- A condição pode ser qualquer expressão lógica
- Use `:` após a condição
- O código dentro deve estar indentado
- Suporta múltiplas linhas de código

Exemplo Prático

```
idade = 20
if idade >= 18:
    print("Você é maior de idade!")
    print("Pode votar e dirigir")
```

O Bloco `elif`: Condições Adicionais

Quando você tem múltiplas condições para verificar em sequência, o `elif` (abreviação de "else if") é sua ferramenta. Ele verifica outra condição apenas se a anterior for **False**. É perfeito para categorizar dados em múltiplas faixas.

1. **Primeira Condição**
   Verifica a condição inicial com `if`
2. **Condições Secundárias**
   Verifica cada `elif` em ordem até encontrar uma verdadeira
3. **Última Opção**
   Se nenhuma condição for atendida, executa `else` (opcional)

O Bloco `else`: A Condição de Fallback

O `else` captura todos os casos que não atenderam nenhuma das condições anteriores. Ele é opcional, mas extremamente útil para garantir que seu programa sempre tenha um comportamento definido.

Exemplo Completo: Sistema de Avaliação

```
nota = 75

if nota >= 90:
    conceito = 'A'
    mensagem = 'Excelente trabalho!'
elif nota >= 80:
    conceito = 'B'
    mensagem = 'Bom desempenho!'
elif nota >= 70:
    conceito = 'C'
    mensagem = 'Desempenho satisfatório'
elif nota >= 60:
    conceito = 'D'
    mensagem = 'Precisa melhorar'
else:
    conceito = 'F'
    mensagem = 'Reprovado'

print(f"Nota: {nota}")
print(f"Conceito: {conceito}")
print(mensagem)
```

Padrões Comuns e Dicas

Padrão: Validação de Entrada

```
usuario = input("Digite seu nome: ")
if not usuario:
    print("Nome não pode estar vazio!")
    exit()
```

Padrão: Faixas Numéricas

```
temperatura = 25
if temperatura < 0:
    print("Congelante!")
elif temperatura < 15:
    print("Frio")
elif temperatura < 25:
    print("Agradável")
else:
    print("Quente")
```

Dica: Evite Aninhamento Excessivo

Use `elif` em vez de `if` dentro de `if` para manter o código limpo e legível.

Erros Comuns a Evitar

Esquecer o Dois-Pontos (:)

Sempre adicione `:` após a condição: `if x > 5:` ✓ vs `if x > 5` ✗

Indentação Incorreta

Use espaços ou tabs consistentemente. Python usa indentação para definir blocos de código.

Comparar com Booleanos

Evite: `if condicao == True:` Use: `if condicao:` é mais pythonico.

Esquecer o `else`

Se houver casos não cobertos, adicione `else` para tratar situações inesperadas.

DESAFIO 4: Pratique suas habilidades no CodeCombat! Acesso disponível no GitHub da disciplina. Data de entrega: 06/ABR/2026 (Aula 5)

Made with GAMMA